



JavaScript Complete Notes for LP Course: Concepts & Techniques

B.tech (Sri Krishna Institute of Technology)



Scan to open on Studocu

JavaScript

COMPLETE NOTES

Prepared By : **Topperworld**

Follow us:

website : **Topperworld.in**
Linkedin : **Topperworld**
Instagram : **Topperworld.in**



Page No:

Date: .../.../...

JavaScript



INDEX

S.No	Title of Topic	Page No
1.	Introduction to Javascript - History of Javascript - Purpose of Javascript - Setting up the development environment	1-4
2.	Basic of Javascript - Variables and Datatypes - Operators and Expressions - Control structure	4-7
3.	Loops - for loop - while loop - Dohwhile loop	7-8
4.	function and Scope - Defining functions - function parameters and return values - Scope and Closures	9-12

5. Javascript Statement

- if
- if ... else if
- if else
- Switch

12-16

6. Arrays and Objects

- working with Arrays
- Creating and manipulating object
- Arrays and object method

16-20

7. DOM manipulation

- Introduction to DOM
- Accessing and Modifying html elements
- Handling Object

20-23

8. Asynchronous Javascript

- Introduction to asynchronous programming
- Callback functions
- Promises and async/await

23-26

9. Error Handling

- Handling exceptions and errors
- Handling Exceptions
- Debugging Techniques

27-29

10.

ES6 and Beyond

- New features of ES6

- let and const
- Arrow function
- classes
- Template Literals
- Destructuring Assignment
- Spread and Rest operators
- Promises
- Modules
- Default Parameters
- Symbol
- Iterator and Generator
- Async / await

29-33

11.

Project Work

33-35

12.

working with dates and times

36-40

- creating and manipulating date objects.
- formatting and displaying dates
- Performing date calculations and comparisons.
- working with time zones.

13.

AJAX and fetch API

40-43

- Introduction to asynchronous programming.
- Making HTTP requests with fetch API

→ Handling responses and data manipulation.

→ working with JSON data

14. Introduction to Javascript framework and Libraries.

43-48

→ Overview of popular framework.

→ Introduction to Libraries

→ Using framework and Library for building interactive web application

→ Pros and Cons of using Javascript framework.

Introduction to Javascript

- JavaScript is a high-level, dynamic and versatile programming language primarily used for web development.
- It is an interpreted, full-fledged programming language that enables interactivity on websites when applied to an HTML document.
- It was created by Brendan Eich in 1995 while he was working at Netscape Communications Corporation. Initially, it was named "Mocha" and later "Live Script" before finally being called "JavaScript".

History of JavaScript

©Topperworld

In 1993, Mosaic, the first popular web browser, came into existence. In the year 1994, Netscape was founded by Marc Andreessen. He realized that the web needed to become more dynamic. Thus, a 'glue language' was believed to be provided to HTML to make web designing easy for designers and part time programmers. Consequently, in 1995, the company recruited Brendan Eich intending to implement and embed Scheme programming language to the

browser. But, before Brendan could start, the company merged with Sun Microsystems for adding Java into its Navigator so that it could compete with Microsoft over the web technologies and platform. Now, two languages were there: Java and the Scripting language.

©Toppersworld

further, Netscape decided to give a similar name to the scripting language as Java's. It led to 'JavaScript'. Finally, in May 1995, Marc Andreessen coined the first code of JavaScript named 'Mocha'. Later the marketing team replaced the name with 'LiveScript'. But due to trademark reasons and certain other reasons, in December 1995, the language was finally renamed to 'JavaScript' from then, JavaScript came into existence.

Purpose of JavaScript

- JavaScript's primary purpose is to enable client side scripting on web pages. This means it allows developers to manipulate the content and behaviour of web pages directly within the user's web browser.
- With JavaScript, we can dynamically update web page elements, validate form inputs and respond to user interactions like clicks, mouse movements, and keyboard input.

• Additionally, Javascript can interact with web servers using AJAX (Asynchronous JavaScript and XML) to fetch data without requiring a page reload.

Setting-up the development environment

1. Text Editor

To start coding in Javascript, we need a development environment set up

Here are the basic

© Topperworld

Choose a text editor or an Integrated Development Environment (IDE) that supports JavaScript syntax highlighting. Examples include Visual Studio Code, Sublime text, or Atom.

2. Web browser

JavaScript runs in web browsers, so we need a modern browser like Chrome, Firefox or Edge to execute our JavaScript code.

3. HTML file

Create a new HTML file that will serve as the container for our JavaScript code. This will include our HTML structure and link to the JavaScript

code using the script tag

4. JavaScript Code

Write the JavaScript code within the script tags in the HTML file or in a separate, js file, which we link to our HTML file.

5. Testing

Open the HTML file in our web browser to see the result of our Javascript code.

As we progress in our Javascript learning journey, we may explore using more advanced development tools frameworks, and libraries to stream our workflow and build complex applications

Basics of JavaScript

JavaScript, being a fundamental programming language, has several essential concepts that form its core.

Let's dive into the basics of JavaScript, including variables and data types, operators and expressions, and control structures -

Variables and Data Types

Variables are containers used to store data values in JavaScript. We can declare variables using the 'var', 'let' or 'Const' keyword.

for Example

©Topperworld

```
Var age = 30;  
let name = 'John';  
Const PI = 3.14;
```

- JavaScript has various data types, including:
- Primitive data Types:
numbers, strings, boolean, null, undefined, and symbols.
- Complex data types
objects (arrays, functions, and Object themselves)

Operators and Expressions

Operators are symbols used to perform operations on variables and values.

- JavaScript supports various types of operators such as arithmetic assignment, comparison

logical etc

Examples

Var $x = 10;$

Var $y = 5;$

Var $Sum = x + y;$

Var $difference = x - y;$

Var $is\ True = x > y;$

- Expressions are combinations of Variables and Values, and operators that evaluate to a single value

for Example;

Var $result = (x + y) * 2;$

Control structures

Control statements or structures allows us to control the flow of our code, making decisions or repeating actions based on conditions

- If else

It allows us to execute a block of code if a

certain condition is true or another block of code if the condition is false

Example

```
Var age = 18;
```

© Topperworld

```
if (age >= 18) {
```

```
    Console.log("You can vote");  
}
```

```
else {
```

```
    console.log("You can't vote");
```

- Loops

for Loop: It allows us to execute a block of data / code repeatedly based on a specified condition

Example

```
for (Var i = 1 ; i <= 5 ; i++) {
```

```
    console.log("Iteration : " + i);
```

```
}
```

While loop: It executes a block of code as long as a specified condition is true.

Example :

```
var Count = 1;
```

```
while (Count <= 5) {
```

```
    console.log ("Count: " + count);
```

```
    count ++;
```

```
}
```

do while loops

© Topperworld

Similar to a while loop, but it executes the code at least once before checking the condition.

```
var num = 1;
```

```
do {
```

```
    console.log ("Number: " + num);
```

```
    num ++;
```

```
}
```

```
while (num <= 5);
```

3. Functions and Scope

Defining functions

In JavaScript, a function is a reusable block of code that performs a specific task. It allows us to encapsulate logic and execute it whenever needed.

functions are defined using the 'function' keyword, followed by a name, a set of parentheses '()' and curly braces '{}' containing the function's body.

Example

```
function greet() {  
    console.log("Hello!");  
}
```

function Parameters and Return values

Parameters are placeholders for values that we can pass to a function when calling it. They enable us to customize the behaviour of the function dynamically. Parameters are defined inside the parentheses of the function declaration.

Example :

```
function greet (name) {  
    console.log ("Hello," + name + "!");  
}
```

Return values ; Functions can also return values using the 'return' statement . The returned value can then be used in other parts of the code .

Example

© Topperworld

```
function add (a,b) {  
    return a+b;  
}
```

Scopes and closures :

Scope refers to the context in which variables and functions are accessible in a code . In JavaScript there are two main types of scope :

• GLOBAL SCOPE

Variables declared outside any function have global scope can be accessed from anywhere in the code .

LOCAL SCOPE

Variables declared inside a function have local scope and are only accessible within that function.

Example of local Scope

```
function my function () {
```

```
    var x = 10;
```

```
    console.log(x);
```

```
}
```

```
console.log(x);
```

Closures: A closure is a powerful feature in JavaScript that allow a function to remember and access its lexical scope even when it's executed outside that scope. This means a function can retain access to its parent function's variables even after the parent function has finished executing.

Example of a closure:

```
function outer function() {
```

```
    var outerVariable = "I am from the outer  
                        function";
```

```
    function inner function () {
```

```
    console.log (Outer Variable);  
  }  
  return innerfunction;  
}
```

```
var closure Example = Outer Function ();  
closure Example ();
```

JavaScript Statement

JavaScript if Statement

It evaluates the content only if expression is true.
The signature of JavaScript if statement is given below.

```
if (expression) {  
  // statement  
}
```

© Topperworld

Ex:-

```
< Script >  
var a = 20;  
if (a > 10) {  
  document.write (" Value of a is greater than 10");  
}  
< / Script >
```

Output: Value of a is greater than 10

JavaScript if ... else if Statement

It evaluates the content only if expression is true from several expressions. The signature of JavaScript if else if statement is given below.

```
if (expression 1) {  
  // Content to be evaluated if expression 1 is true  
}  
else if (expression 2) {  
  // Content to be evaluated if expression 2 is true  
}  
else if (expression 3) {  
  // Content to be evaluated if expression 3 is true  
}  
else {  
  // Content to be evaluated if no expression is true  
}
```

Example :-

```
<script>  
var a = 20;  
if (a == 10) {  
  document.write("a is equal to 10");  
}  
else if (a == 15) {  
  document.write("a is equal to 15");  
}  
else if (a == 20) {  
  document.write("a is equal to 20");  
}
```

```
}  
else {  
document.write ("a is not equal to 10, 15 or 20");  
}  
</script>
```

©Topperworld

Output : a is equal to 20

JavaScript If... else Statement

It evaluates the content whether condition is true or false. The syntax of Javascript if- else statement is given below

```
if (expression) {  
// Content to be evaluated if condition is true  
}  
else {  
// Content to be evaluated if condition is false  
}
```

Example :-

```
var a = 20;  
if (a % 2 == 0) {  
document.write ("a is even number");  
}  
else {  
document.write ("a is odd number");  
}  
</script>
```

Output : a is even number

JavaScript Switch

The JavaScript Switch statement is used to execute one code from multiple expressions. It is just like else if statement that we have learned in previous page. But it is convenient than if-else-if because it can be used with numbers, characters etc.

The signature of JavaScript switch statement is given below

Switch (expression) {

Case value 1:

code to be executed;

break;

Case value 2:

code to be executed;

break;

.....

default:

code to be executed if above values are not matched;

}

Example

```
< Script >
Var grade = 'B';
Var result;
Switch (grade) {
  Case 'A' ;
  result = "A Grade" ;
  break ;
  Case 'B' ;
  result = "B Grade" ;
  break ;
  Case 'C' ;
  result = "C Grade" ;
  break ;
  default ;
  result = " No Grade" ;
}
document. write (result);
</ Script >
```

©Topperworld

Output: B grade

Arrays and Objects

Working with Arrays

Array are data structures in JavaScript used to store collections of elements. They allow us to group multiple values into a single variable.

Arrays are created using square brackets '['']' and each element is separated by comma.

Example :

```
Var fruits = ['apple', 'banana', 'orange'];
```

- Accessing elements : We can access individual elements in an array using their index, which starts from 0.

Example :

```
Var first fruit = fruit[0];
```

- Modifying elements : We can modify elements in an array by assigning new values to their corresponding indices

Example :

```
fruits [1] = 'grape';
```

- Array methods : JavaScript provides various built in methods to manipulate arrays, such as 'push()', 'pop()', 'shift()', 'unshift()', 'splice()' and more.

Creating and manipulating objects

Objects are collections of key-value pairs. Each

Key is a property that represents a name or identifier and each value is the associated data.

Objects are enclosed in curly braces '{ }'

Example :

© Topperworld

```
Var person = {  
    name : 'John',  
    age : 30,  
    city : 'New York'  
};
```

- Accessing Object properties : We can access properties of an object using dot notation or square brackets.

Example :

```
Var person Name = person.name;  
Var person Age = person['age'];
```

- Modifying object properties

We can modify object properties by assigning new values to them

Example :

```
var student = {  
    name : 'Alice',  
    age : 25  
    address : {  
        street : '123 Main St',  
        city : 'Los Angeles'  
    }  
};
```

Array and Object methods

Arrays and objects have built in methods that allow us to perform various operations efficiently.

Some common methods include:

• Array methods

'push()', 'pop()', 'shift()',
'unshift()', 'splice()', 'concat()',
'slice()', 'indexOf()', 'forEach()',
'map()', 'filter()', and more

• Object methods

'Object.keys()',
'Object.values()',
'Object.entries()';

and 'object.assign ()'

Example of using array and object methods

```
Var numbers = [ 1, 2, 3, 4, 5 ];
```

```
numbers.push (6);
```

```
numbers.pop ();
```

©Topperworld

```
Var student = {  
    name : 'Bob',  
    age : 22,  
    city : 'Chicago'
```

```
} ;
```

```
Var keys = Object.keys (student);
```

```
Var values = Object.values (student);
```

DOM Manipulation

Introduction to the Document Object Model

The Document Object Model (DOM) is a programming interface for HTML and XML documents.

It represents the structure of a web page as a hierarchical tree of objects.

Each element in the HTML document is represented as a node in the DOM tree,

allowing developers to interact with and manipulate the content and structure of a web page dynamically

Accessing and Modifying HTML Elements

JavaScript can be used to access and modify HTML elements in the DOM, enabling dynamic updates to the web pages content and appearance

- Accessing elements : We can use various methods to access elements by their ID, class, tag name, or other attributes

Examples :-

```
Var heading Element = document . get Element By  
Id ('heading');
```

```
Var elements With class = document . get Element  
By class ("highlight");
```

```
Var all paragraphs = document . get Element  
By TagName ('p');
```

- Modifying elements : Once we have access to an element, we can change its content, attributes or styles using JavaScript.

Examples :

Topperworld

```
heading Element. text content = 'New Heading';
```

```
Var image Element = document. get Element By Id  
(: my Image);
```

```
image Element. src = 'new-image.jpg';
```

```
Var. paragraph Element = document. get Element  
By Id (' my Paragraph');
```

```
paragraph Element. style color = 'blue';
```

Handling Events

Events are actions or occurrences that happen in the web page, such as clicking a button, moving the mouse, or pressing a key. Java Script allows us to handle these events and perform specific actions in response.

Adding event listeners : We can attach event listeners to elements to listen for specific events and trigger functions when those event occurs.

Example

```
Var my Button = document. get Element By
```

Id ('My Button') ;

```
my Button . add Event listener ( ' click' ,  
    function ( ) {  
        alert ( ' Button clicked ! ' ) ;  
    } ) ;
```

```
document . add Event Listener ( ' keydown' ,  
    function ( event ) {  
        console . log ( ' Key pressed : ' ,  
            event . key ) ;  
    } ) ;
```

DOM Manipulation is a crucial aspect of front-end web development allowing developers to create interactive and dynamic user experiences.

With JavaScript and the DOM, we can build engaging web applications and respond to user interactions effectively.

Asynchronous JavaScript

Asynchronous programming in JavaScript allows tasks to run independently without blocking the execution of the rest of the code. It's essential for handling time consuming operations, such as

making APIs requests, reading files or waiting for user Input, without freezing the user interface.

Callback functions

©Topperworld

A Common approach to asynchronous programming in Javascript is using callback functions.

A Callback is a function that is passed as an argument to another function and executed later, usually when an asynchronous task is complete.

```
function fetch Data from Server (callback) {  
  Set Time out ( function() {  
    var data = { name : 'John',  
                age : 30 };
```

```
    callback (data);  
  }, 1000);  
}
```

```
function process Data (data) {  
  console.log ( 'Data received', data);  
}
```

```
fetch Data from Server (process data);
```

Promises and async/await

Promises are an improved way of dealing with asynchronous operations in JavaScript.

They represent a value that may be available now or in the future, either resolved (Successful) or rejected (failed).

```
function fetch Data from Server ( ) {
```

```
    return new Promise (function (resolve ,  
                                reject) {
```

```
        Set Timeout (function ( ) {
```

```
            Var data = { name : 'John', age : 30 };
```

```
            resolve ( data );
```

```
        } , 1000 );
```

```
    } ) ;
```

```
fetch Data from Server ( )
```

```
    then (function (data) {
```

```
        Console.log ('Data received: ' , data);
```

```
    } )
```

```
    Catch (function (error) {
```

```
        Console.error ('Error : ' , error.message);
```

```
    } ) ;
```

Async/await is a modern syntax introduced in ES2017 that simplifies working with promises. It allows us to write asynchronous code in a more synchronous-like manner, making it easier to read and maintain.

©Topperworld

```
async function fetchData() {  
    try {  
        var data = await fetchDataFromServer();  
        console.log('Data received: ' + data);  
    }  
  
    catch (error) {  
        console.error('Error: ' + error, message);  
    }  
}  
  
fetchData();
```

Asynchronous JavaScript is vital for handling time-consuming tasks and ensuring a smooth user experience in web applications.

Callbacks, promises and 'async/await' are powerful tools that enable developers to write efficient and responsive code for handling asynchronous operations.

Error Handling

Handling Exceptions and errors

Error handling in JavaScript is the process of managing unexpected situations or exceptions that may occur during the execution of our code. Error can be caused by various factors, such as incorrect input, network issues, or logical mistakes in the code.

Handling Exceptions

In JavaScript, exceptions are raised when an error occurs during the execution of a statement or a function. We can handle exceptions using 'try', 'catch', and optionally 'finally' blocks.

```
try {  
    var result = Some function ();  
}  
catch (error) {  
    console.error('An error occurred:',  
        error.message);  
}  
finally {  
    console.log('The try-catch block has  
        finished');  
}
```

By using the try-catch block, we can gracefully handle error and prevent our application from crashing.

Debugging Techniques

Debugging is the process of identifying and fixing issues in our code.

Java Script provides various techniques to debug our code and find the root cause of errors.

Console.log() The Simplest way to debug is by adding 'Console.log()' statements in our code to output variables or messages to the browser's console.

Console.error() Use 'Console.error()' to print error messages to the console explicitly. This is helpful for highlighting critical issues in our code.

Breakpoints: Modern browsers offer developer tools that allow us to set breakpoints in our code. A break point stops the execution of our code at a specific line, enabling us to inspect variables and step through the code to understand its flow better.

• Debugger Statement We can insert the 'debugger' statement in our code, and when the code is executed, it will pause at that line if the developer tools are open.

• Stack Trace When an exception occurs the stack trace provides information about the sequence of function calls leading to the error. It helps us to track the flow of execution and identify the source of the error.

Browser Developer Tools Use the browser's developer tools to inspect and debug our code. They offer features like Console logs, network monitoring, performance analysis and more.

ES6 and Beyond

New features introduced in ECMAScript 6 and later versions

ES6, short for ECMAScript 6, is the sixth edition of the ECMAScript standard, which is the specification that defines the JavaScript language.

It was released in 2015 and introduced several new features and enhancements to JavaScript.

©Topperworld

Since then, newer versions of ECMAScript have been released, each bringing additional improvements and features.

Here are some notable features introduced in ECMAScript 6 and beyond.

1. Let and Const

Introduced in ES6, 'let' and 'const' are block-scoped variables. 'let' allows us to declare variables that can be reassigned, while 'const' declares constants whose value cannot be changed.

2. Arrow functions

Arrow functions provide a concise syntax for writing functions in JavaScript. They have implicit returns and lexically bind the value of 'this'.

Classes

ES6 introduced class syntax for creating

objects, supporting constructors methods and inheritance. It is more familiar to developers coming from other object Oriented programming languages.

Template Literals

Template literals allow us to create multiline strings and embed expression within backticks (` `) instead of using single or double quotes

Destructing Assignment

Destructing assignment allows us to extract values from arrays or objects into separate variables using consize syntax

Spread and Rest Operators

The spread operator (`...`) spreads the elements of an array into individual elements, while the rest parameter (`...`) collects individual arguments into an array.

Promises

promises provide a better way to work with asynchronous operations in JavaScript, making it easier to handle and Success

and failure conditions.

Modules

© Topperworld

ES6 introduced a standardized module system using 'import' and 'export' statements, allowing for better code organization and reuse.

Default parameters

We can define default values for function parameters if no value is passed or if the argument is 'undefined'.

Symbol

Symbols are unique and immutable data types introduced in ES6, useful for creating non-enumerable properties and avoiding naming collisions.

Iterators and Generators

Iterators allow us to loop through data structures while generators are functions that can pause and resume their execution, useful for creating iterations.

Enhanced Object Literals

ES6 introduced enhancements to Object literals, allowing us to define method and computed property names.

Async/ Await

Introduced in ES2017, 'async/await' is a powerful syntactic feature that simplifies working with promises and handling asynchronous code in a more synchronous like manner.

These features significantly improve JavaScript's readability, maintainability and expressiveness making it easier for developers to build complex applications with fewer lines of code and better organization.

Project Work

Building a Small JavaScript application or website

Project Work in JavaScript involves building a small applications or website using for coding skills and knowledge of Java

Script. It is an excellent way to apply what we have learned and gain hands-on experience in real-world scenarios.

Here are some key steps and considerations when working on a JavaScript project.

Define the project Scope

Topperworld

Decide on the type of project we want to build. It could be a simple to-do list app, a weather app, a calculator, a portfolio website, or anything that interests us.

Clearly define the features and functionality we want to implement in our project.

Plan and Design

Create a rough sketch or wireframe of our applications user interface. Decide on the layout, components and how users will interact with it.

plan the project's structure, including the different JavaScript functions, modules and files we'll need.

Set up the development environment

Ensure we have a text editor or IDE of our choice and a modern web browser.

Set up a version control system like Git to track changes and manage our code effectively.

Implement and functionality

Write the necessary HTML, CSS and Javascript code to create the desired features.

Pay attention to code organization and best practices to make our code readable and maintainable

Test and Debug

Test the application thoroughly to identify and fix any bugs or issues.

Use debugging techniques like 'console' log() or browser developer tools to inspect variables and troubleshoot problems.

Working with Dates and Times

Working with dates and times is a common task in web development, as it enables us to display time sensitive information, schedule events and perform various data related calculations.

©Topperworld

In this chapter, we will explore how to create, manipulate, format and display dates in JavaScript.

Creating and manipulating Date objects

In JavaScript, we can work with dates using the Date object, which provides methods for creating and manipulating dates.

To create a new Date object, we can use the new Date () Constructor.

```
Var Current Date = new Date ( );  
console.log ( Current Date );  
// output the current date  
and time
```

We can also create specific dates by passing year, month, day, hr,

minute, second and millisecond values to the Date constructor.

```
Var Specific Date = new Date ( 2023 , 6 , 15, 12,
                               30, 0, 0 );
```

```
Console.log (Specific Date);
```

// Output Sun Jul 15 2023 12:30:00

To manipulate dates, we can use various methods such as 'setfull year (), set Month (), set Date (), set Hours (), set Minute (), set Second () and set Millisecond ()'.

```
Var data = new date ( );
date.set full Year ( 2023 );
date.set Month ( 7 );
date.set Date ( 23 );
console.log (date);
```

// Output : web Dec 05 2023 09:00:00

formatting and displaying dates

To display data in a more readable format, we can use various methods to extract specific data and components and format them as needed.

```
Var date = new Date ( );
Var year = date.get full year ( );
Var month = date.get Month + 1;
    // Adding 1 to match the actual Month
Var day = date.get Date ( );
Var hours = date.get Hours ( );
Var minutes = date.get Minutes ( );
Var Seconds = date.get Seconds ( );
Console.log (year + '-' + month + '-' +
              day + '-' + hours + ':' + minutes
              + ':' + Seconds);
// Output : Current date and time in
            'yyyy-mm-dd' 'HH:mm:ss'
            format
```

©Topperworld

Alternatively, we can use 'toLocaleString()' method to get a localised date and time representator.

```
Console.log (date.toLocaleString ( ));
            output : localised date and
                    time representations
```

Performing Date calculations and Comparisons

JavaScript provides several methods to perform date calculations and comparisons. We can use methods like get Time () to get the time stamp of a date and

perform mathematical operations on dates

```
Var date 1 = new Date ( 2023 , 6 , 15 ) ;
```

```
Var date 2 = new Date ( 2023 , 8 , 20 ) ;
```

```
Var difference in Milliseconds = date 2 . get Time  
() - date 1 get Time ( ) ;
```

```
Var difference in Days = difference in Milliseconds  
/ ( 1000 * 60 * 60 * 24 ) ;
```

```
Console . log ( difference in Days ) ;
```

// Difference in days between the 2 dates

We can also compare dates using comparison operators like (< , > , = , >=)

Working with Time Zones

Dealing with time zones can be challenging, as different locations have different time effects. JavaScript Date object uses the host system's time zone by default, but we can work with different time zones using libraries like Moment.js or the built-in 'toLocaleString()' method with appropriate options.

```
Var date = new Date ( ) ;
```

```
console . log ( date . to local string ( 'en - us' ,
```

```
{ timezone : 'America / NewYork' } )  
);
```

// Output : Date and time in New York time zone.

© Topperworld

AJAX and Fetch API

Asynchronous JavaScript and XML (AJAX) and fetch API are powerful tools that allow web developers to create dynamic and interactive web applications.

In this chapter, we will explore asynchronous programming, making HTTP requests with the fetch API, handling responses, manipulating data and working with JSON data.

Introduction to Asynchronous Programming

In traditional programming, code executes in a sequential manner, one line after another. Asynchronous programming, on the other hand, allows tasks to run independently of the main program flow. This is essential for tasks that may take time to complete, such as making HTTP requests to external servers or

fetching data from a database

Asynchronous programming is crucial for building responsive web applications as it prevents long-running tasks from blocking the user interface and provides a 'smoother user experience'.

Making HTTP requests with Fetch API

The fetch API is a modern and native JavaScript API that simplifies making HTTP requests. It provides a simple and easy to use interface to fetch resources (such as JSON data, images or text) from the server.

Here's an example of making a GET request using the fetch API.

```
fetch('https://api.example.com/data')  
  .then(response => response.json())  
  .then(data => console.log(data))  
  .catch(error => console.log('Error  
    fetching data', error));
```

In this example, fetch is used to make a GET request to the specified URL. The response is converted to JSON format using the

JSON() method. The data is then logged to the console.

Handling responses and data manipulation

Once we receive the response from the server we can handle it and manipulate

for example : We can extract specific information, display it on the web page, or use it for other tasks.

©Topperworld

```

• fetch ('https : // api . example . com / data')
  • then (response ⇒ response . json ())
  • then (data ⇒ {
      // manipulate and use data
      console . log (data) ;
      document . get Element ById ('result') . text
      Content = data . message ;
    })
  catch . (error ⇒ console . error ('Error
    fetching data , error));

```

In this example, the data received from the server is used to set the content of an element with the ID 'result' on the web page.

Working with JSON data

JSON (JavaScript Object Notation) is a popular data format used for exchanging data between a web server and web client.

The fetch API automatically converts the response to JSON using the 'JSON()' method, making it easy to work with JSON data.

```
fetch('https://api.example.com/data')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(
    'Error fetching data:', error));
```

JSON data is usually structured as key-value pairs, making it easy to access and manipulate specific data elements.

Introduction to JavaScript Frameworks and Libraries

JavaScript frameworks and libraries are essential tools for web developers, as they provide a structured and efficient way to

build complex and interactive web applications.

In this chapter, we will explore popular JavaScript frameworks, introduce libraries, discuss their role in building interactive web applications, and examine the pros. and cons of using JavaScript frameworks.

Overview of popular JavaScript frameworks

JavaScript frameworks are comprehensive tools that provide a complete structure and set of functionalities for building web applications.

Some of the most popular JavaScript frameworks include React, Angular and vue.

React

Developed and maintained by facebook
React is a declarative component based JavaScript library for building user interfaces. It allow developers to create reusable UI components, making it easier to manage complex applications.

Angular

Developed and maintained by Google.

Angular is a comprehensive JavaScript frameworks that provides a complete solution for building large scale applications. It offers powerful features such as data binding, dependency injection and routing.

Vue

©Topperworld

Vue is a progressive and user friendly JavaScript framework that allows developers to build interactive user interface. It provides a gentle learning curve and can be integrated incrementally into existing projects.

Introduction to Libraries

JavaScript libraries on the other hand are collections of pre-written code that simplify and speed up development tasks. They are focused on providing specific functionalities making them lightweight and easy to use.

JQuery

JQuery is one of the most popular JavaScript libraries. It simplifies DOM manipulation, event handling, and

AJAX calls, making it easier to work with page elements and interact with servers.

Lodash

Lodash is a utility library that provides a wide range of helper functions to work with arrays, objects, strings and more. It improves code efficiency and readability by offering methods for common data manipulation tasks.

Using frameworks and libraries for building interactive web applications

JavaScript frameworks and libraries are essential for building interactive and dynamic web applications. They offer a structured architecture and a wealth of tools that enable developer to manage complex data, handle user interactions, and update air interface efficiently.

Using frameworks and libraries allows developers to save time and effort by leveraging pre built solutions for common tasks. This, in turn, improves development speed and helps deliver more robust and feature rich applications.

Pros and cons of using JavaScript frameworks

Using JavaScript can be beneficial for many reasons

Structured Architecture

frameworks provide a structured architecture that promotes organization and maintainability

Reusability

Components and frameworks can be reused across different parts of the application, programming code reuse.

Efficiency

frameworks offer pre-optimised solutions for common tasks improving development efficiency

However there are also some cons to consider

Learning curve

Some frameworks have a steep learning curve, especially for beginners.

Complexity

Large frameworks can introduce unnecessary complexity for smaller projects.

Performance

Some frameworks may add overhead to the application, affecting performance.